
Geant4の基本構造と必須クラス

Geant4 11.3 準拠

Geant4講習会資料: HEP/NP/Space 2025



KEK

大学共同利用機関法人

高エネルギー加速器研究機構

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

目次

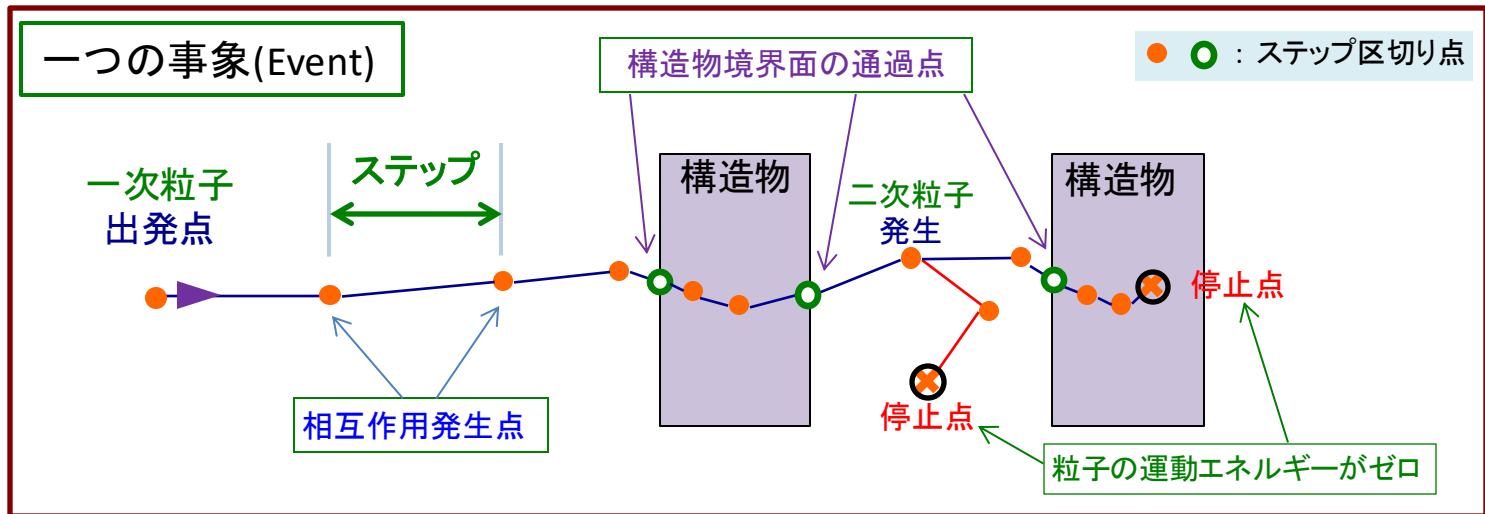
- 基本的なオブジェクト
- Geant4のプログラム構造
- ユーザクラス実装の予備知識
- Geant4アプリケーション作成の第一歩

基本的なオブジェクト

Geant4のシミュレーション手法

前の講義でシミュレーションは以下のように進むことを述べた

- 与えられた粒子進路(Track)に沿って、ステップ(Step)単位で粒子輸送が行われる
 - 相互作用の発生、構造物の境界面を通過ごとにステップが切られる
 - 従って、ステップ長さはGeant4が自動的に決定（ユーザがコントロールすることも可能）
- シミュレーションは事象(Event)単位で行われる
 - 事象内の全粒子の輸送が完了したら一つ事象のシミュレーションが完了
- ユーザはステップ情報、タリー情報などでシミュレーションの結果を取得できる
- 通常、シミュレーション実行では一つのジョブ内で多数の事象を一括処理する



基本的なオブジェクトとは？

- Geant4はオブジェクト指向言語C++で作成されたシミュレーション・ツール・キット
 - ツールキットはシミュレーションに必要な機能をC++**クラス**のライブラリで提供
 - ユーザはライブラリとして提供されたクラスの中で、特にユーザ用に用意されたユーティリティ・クラスを用い、目的のシミュレーションのプログラムを作成
- 従って、ユーザはGeant4が用意するユーザ用ユーティリティ・クラス、およびそれと関連が深いクラスの意味を知っておくことが必要
- 以下ではユーザが知るべき基本的なクラスの意味と機能のまとめ
 - ここで説明する以外にも、必要に応じて他の講義の中で新たなクラスを解説する
- 本講義ではクラス関連の用語を無理に**日本語訳をしない**
 - 基本的に英語を用いる
 - 必要に応じて対応する日本語を示す

[注]

以下のスライドではあるクラスのオブジェクトを以下のように記述する

G4Run**オブジェクト** ； **G4Run**クラスの**オブジェクト**という意味

Runとは？

- 同一の条件のもとで一連の事象をシミュレーションすることを'Run'を実行するという
 - 'Run'という言葉は、粒子加速器実験で、粒子ビームの照射開始から終わりまでの一連の測定に対して名付けられたもの
- 'Run'の開始時に以下の設定が自動的に行われる
 - シミュレーションで考慮される構造物(計測機器など)の幾何情報('Geometry')の最適化
 - 構成物質と粒子の相互作用の反応断面積の計算
 - 二次粒子生成条件の設定 (ユーザが設定した値に従って)
- 一つの'Run'の実行中は、以下のシミュレーション条件をユーザが変更することは不可
 - 構造物のセットアップ
 - シミュレーションで考慮する物理過程
- 一つのRunに関連する情報は'G4Run'クラスで表現される
 - 'G4Run'オブジェクトは一つの'Run'でのシミュレーションの全体情報を保持する
- 'G4RunManager'オブジェクトは'Run'の間の全てのシミュレーション処理を制御
 - [注] G4RunManagerはユーザがアプリケーションを簡単に作れるように用意されたクラス
多くのアプリはこのクラスを使い、main()プログラムを書くだけでシミュレーションを完成可能
しかし、必要に応じてこのクラスを使わずにアプリを作成することも可能
- 'G4UserRunAction'オブジェクトは'Run'実行中にユーザが必要に応じてシミュレーションに介入するためのユーザ・フック

Eventとは？

- ‘Event’ (事象) とは ‘Primary Particles’ (初期粒子 : 一次粒子ともいう) が構造物に入射したことで生じさせる物理現象の全てをさす
 - 前述のRunは複数の ‘Events’ の集合
- ‘Primary Particles’ の輸送 (Tracking) が開始されることで一つの ‘Event’ は始まる
- ‘Tracking’ 中に生成された2、3、4....次粒子が起こす物理現象も ‘Event’ に含まれる
 - 生成された2、3、4....次粒子をまとめて「二次粒子」とよぶ
- 1次粒子および、2次粒子が全て ‘Tracking’ されることで一つの ‘Event’ 処理が完了
- ‘G4Event’ オブジェクトは一つの ‘Event’ を表現し、以下の情報を保持する
 - 一次粒子の情報及びそれらの空間始点情報 (初期情報)
 - 生成された二次粒子のシミュレーション経過情報 (Trajectoryとよぶ) のなど
- ‘G4EventManager’ オブジェクトが一つの ‘Event’ のシミュレーションを制御
- ‘G4UserEventAction’ オブジェクトは ‘Event’ 処理中に必要に応じてユーザがシミュレーションに介入するためのユーザ・フック

Trackとは？

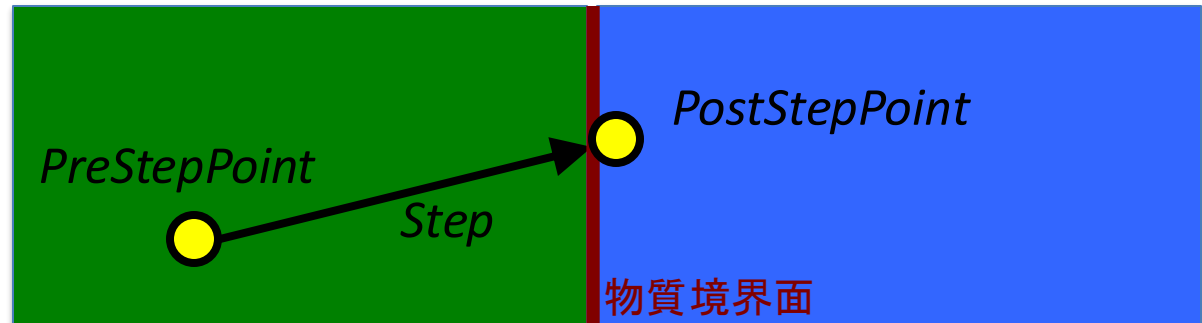
- Geant4は粒子輸送('tracking')をStep(後述)単位で進める
- Trackingは以下の条件で終了する
 - 粒子が設定されている幾何学的な全空間(ワールド)の外に出た時
 - 粒子が崩壊、非弾性散乱等で消滅した時
 - 粒子の運動エネルギーがゼロ(すなわち静止状態)
 - ユーザが強制的に粒子のtrackingを停止した時
- 'Track'はtracking中の粒子の現時点でのStep情報を保持するとともに、tracking開始時点からの現在までの総step数、経過時間、trackの長さなどの情報を保持
 - Trackは現時点の'step'情報のみ保持 — 過去の'step'情報は'G4Trajectory'オブジェクトが保持
 - 情報の詳細と取得方法については以下のwebページ(Section: Tracks and steps)を参照
<https://geant4-userdoc.web.cern.ch/UsersGuides/FAQ/html/index.html>
- Trackの保持する情報は、そのtrack処理の終了時に破棄
 - Track情報を保持したい場合は、eventが持つ'G4Trajectory'オブジェクトに書き加える
- Trackは'G4Track'オブジェクトで表現
- 'G4TrackingManager'オブジェクトは粒子のtrackingを制御
- 'G4UserTrackingAction'オブジェクトは tracking処理中に必要に応じてユーザが介入するためのユーザ・フック

Stepとは？

- Stepはtrackingの最小単位
 - 粒子を一つのstepだけ進めることをsteppingとよぶ
- Stepの始点をPre-step point、終点をPost-step pointとよぶ
- Stepは'G4Step'オブジェクトで表現、Stepの始点と終点は'G4StepPoint'オブジェクトで表現される
 - G4stepオブジェクトはstepの長さ、エネルギー損失などstep全体に関する情報を保持
 - G4StepPointオブジェクトはstep前後での粒子の物理状態、位置などの情報を保持
 - 情報の詳細と取得方法については以下のwebページ(FAQ: 4. Tracks and steps)を参照
<http://geant4.web.cern.ch/geant4/UserDocumentation/UsersGuides/FAQ/html/index.html>
- 'G4SteppingManager'オブジェクトは粒子の一つのsteppingを制御
- 'G4UserSteppingAction'オブジェクトは各stepでの情報を得るためのユーザ・フック
 - このフックを用いてユーザはtracking過程の最も詳細な情報にアクセスが可能

粒子が物質境界面まで輸送されると、pre-step pointは境界面前の物質、post-step pointは境界面後の物質に属するようになる

➡ step pointのrelocationとよぶ

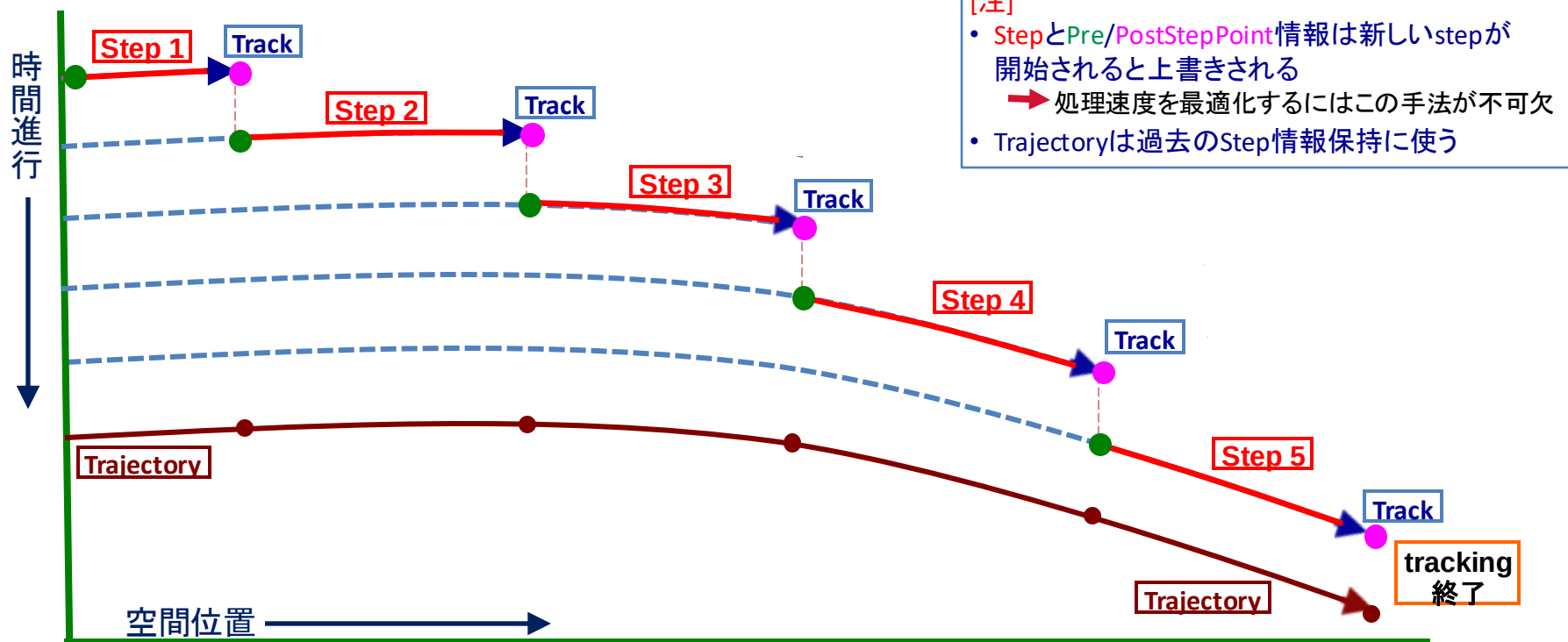


Particleとは？

- Geant4で定義されている様々な素粒子、原子核など(**particles**) を表現する
- 以下のタイプが定義されている
 - 通常の粒子: 電子、陽子、中性子、 π 粒子、 μ 粒子、など
 - 共鳴による短寿命な粒子: Δ 、 N 、 ρ 、 ω 、など
 - 原子核: α 、重イオン、など
 - クォーク: u , d , s , など
- これらのparticlesは以下のようにグループ分けして管理されている
 - レプトン
 - ハドロン → バリオン、メソン、イオン
 - ボゾン および 短寿命粒子
- それぞれのparticleは'**G4ParticleDefinition**'クラスから継承されて定義されている
 - class **G4Electron** : public G4ParticleDefinition
 - class **G4Gamma** : public G4ParticleDefinition
 -
- それぞれのparticleはどのような物理相互作用を起こすかの情報を保持
- particlesはGeant4アプリケーション起動時に**自動的に生成** (一部のイオンを除いて)
- 定義されているparticleは'**G4ParticleTable**'オブジェクトを通してアクセスできる
- **G4Track**オブジェクトは粒子タイプ情報として**G4ParticleDefinition**オブジェクトを保持

Trackingに関連するオブジェクトのまとめ

■ Track, step, trajectoryの指す空間点の時間変化



■ Track, step, trajectoryの保持する情報

Track (---➡) : track#, 積算track長、vertex情報、particle情報、stepへのポインター、.....

Step (—) : step#, step長、energy deposit, pre/postpointへのポインター、....

PreStepPoint (●) : } 位置情報, momentum情報, volume情報, touchable,....

PostStepPoint (●) : }

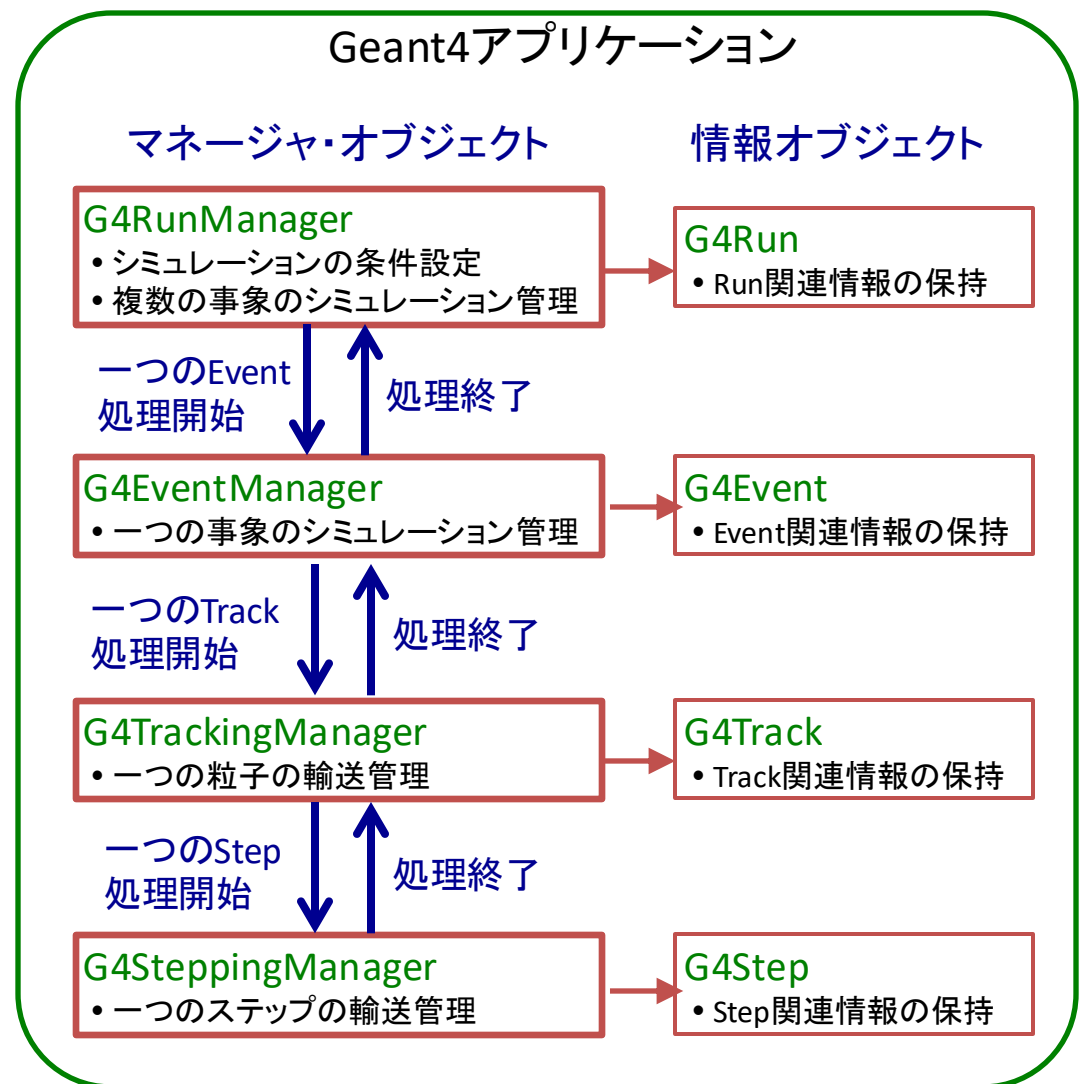
Trajectory (—) : オプション設定して、Step/StepPointから選択情報をこのオブジェクトに書き出す

[注] track長、step長は多重散乱を加えない純空間的な移送距離

Geant4のプログラム構造

Geant4アプリケーションの基本構造

- シミュレーションは4つの階層構造を通して自動進行
- 階層内でのシミュレーション進行の管理はマネージャ・オブジェクトが行う
- それぞれの階層で作られた情報は情報オブジェクトに保持
- ユーザは最上位のマネージャであるG4RunManagerに必要な初期情報を渡すだけでシミュレーションを実行できる
 - 次のスライド参照



ユーザは何をしなければならないか？

- ツールキットが提供するC++クラスを使って、次の3つの初期情報を *G4RunManager* に通知するだけでシミュレーションを実行できる

1) シミュレーション空間に存在する構造物の情報

- 構造物を構成する物体、物質の定義

2) 考慮する物理相互作用

- 考慮すべき粒子、物理相互作用／モデルの定義
- 二次粒子生成閾値の設定

3) 事象の初期条件

- 一次粒子(一つの事象をシミュレーション開始する際に、初期に存在する粒子)の初期条件(場所、エネルギー、等)の定義

[注] 定義した構造物をGeant4に描画させるだけであれば(シミュレーションの実行は行わない)、事象初期定義は不必要

- 上記初期情報の *G4RunManager* への通知はユーザアプリケーションの *main()* 関数の中で行う

- すなわち、ユーザは *main()* を自ら書く必要がある

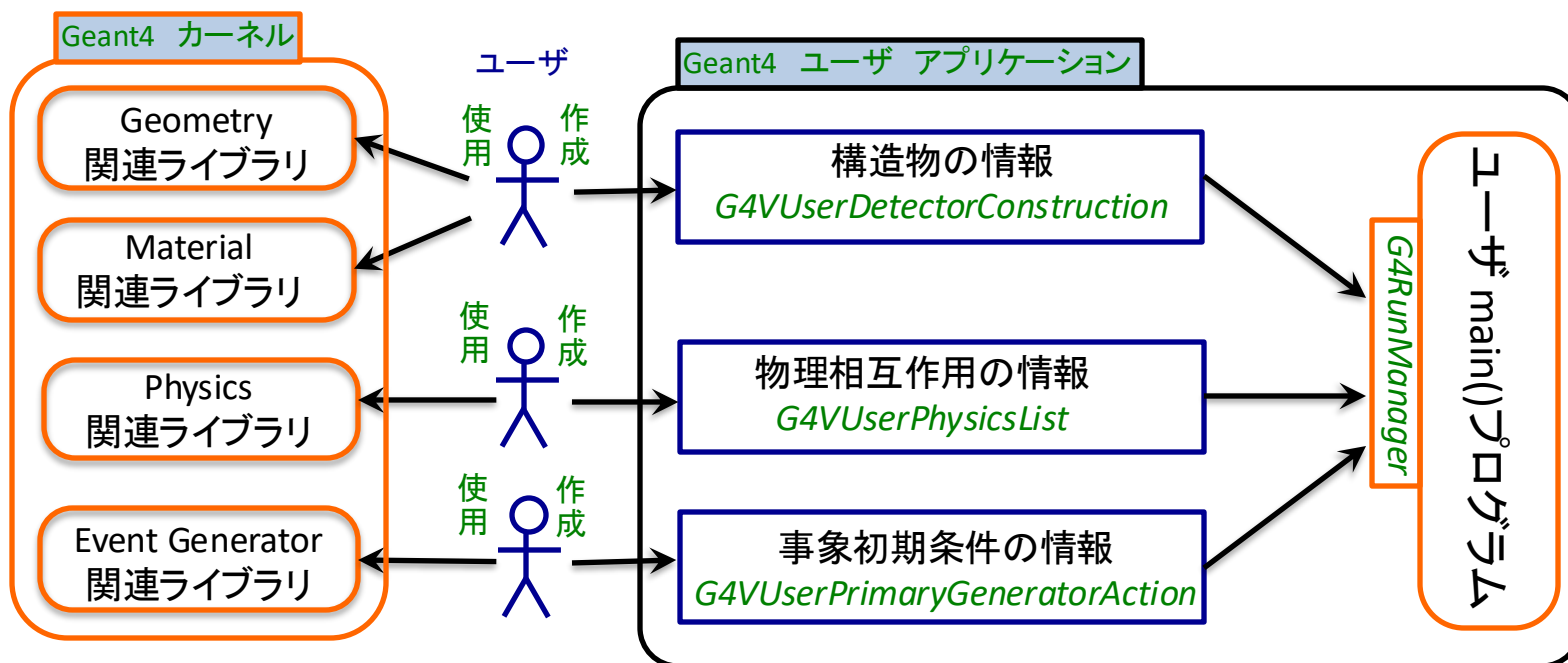
← 実際には、Geant4が提供する例題(サンプルプログラム)からその *main()* をコピーし、必要に応じてそれに変更を加えるだけでOK

ユーザクラスとは？

- ユーザクラスとはユーザがシミュレーション初期情報をGeant4カーネルに伝える時に使用する基底クラスの総称
- ユーザがアプリケーションを作成するには、このクラスの基本的な使い方さえ理解すればよい
- ユーザは以下の3つのユーザクラスからオブジェクトを生成して`G4RunManager`に渡せば、シミュレーションが開始可能:

- 1) 構造物 → `G4VUserDetectorConstruction`
- 2) 粒子と物質の相互作用 → `G4VUserPhysicsList`
- 3) 事象初期条件 → `G4VUserPrimaryGeneratorAction`

ユーザはこれらのクラスを継承して3つの独自のクラスを定義し、その中に3つシミュレーション初期情報を記述する



ユーザクラスの種類

■ ユーザクラスはその機能で以下の二つに分類

1. 初期化必須クラス(Mandatory Initialization Classes)

[機能] シミュレーションの初期化時に必要となる情報を登録

- G4VUserDetectorConstruction 構造体情報
- G4VUserPhysicsList 粒子と物質の相互作用
- G4VUserPrimaryGeneratorAction 事象の初期条件

G4VUserActionInitializationを
経由して登録（下記参照）

2. ユーザアクション・クラス(User Action Classes)

[機能] シミュレーションに必要な情報を登録（必要に応じて使用）

- G4VUserActionInitialization action(ユーザフック)登録は必ずこのオブジェクトを経由
- G4UserRunAction 'Run'処理へのaction (ユーザフック)
- G4UserEventAction 'Event'処理へのaction (ユーザフック)
- G4UserTrackingAction 'Track'処理へのaction (ユーザフック)
- G4UserSteppingAction 'Step'処理へのaction (ユーザフック)
- G4UserStackingAction Track処理順序を制御するaction (ユーザフック)

シミュレーション結果の取得

- Geant4は前述の3つの情報が*G4RunManager*に渡されれば、あとは自動的に全事象の粒子輸送(*tracking*)を実行
 - デフォルトは最小情報のみを実行端末に書き出した後、シミュレーションを終了
 - ユーザが望むシミュレーション結果情報の取得には、独自のコード(通常C++) を書くか、あるいは標準的に用意されている対話型コマンド(UIコマンド)を使用することが必要
 - 結果情報の取得には以下の*二通りの方法*が用意されている
 - ユーザフック(*G4UserTrackingAction*, *G4UserSteppingAction*等)の使用
 - ・ フックからシミュレーション結果のあらゆる情報を入手可能
 - ・ 手法は単純であるが、ユーザがコードを書くことが必要
 - ・ 個々の粒子が輸送される経過を知りたい場合に使う (例: HEPの測定器シミュレーション)
 - スコアラー(*scorer*、*scoring*ともよぶ)を使う
 - ・ スコアラー関連コマンド(UIコマンド)を用いて必要情報をファイルに書き出す
 - ・ 特定の物体を指定し、そこでの物理量をスコアリングするC++コードを書く
 - ・ 複数事象の積算で得られる情報を知りたい場合に使う (例: 医療での線量計算)
- [注] 「スコアリング」手法は以後の講義で触れる

ユーザ支援機能

- ユーザが前述の基本情報を記述する際に使用できる、便利なC++クラスが用意されている。また、それ以外にも、シミュレーション作成にあたって、ユーザを支援する様々なクラスが多く用意されている
- 例：
 - ユーザが定義した構造物の可視化
 - Trackingされた粒子のトラジェクトリ可視化
 - シミュレーション処理の各段階でユーザが介入できる多種のコマンド
 - (グラフィカル)ユーザ・インターフェイス
 - マクロ作成
 - その他

[注] これらの機能については後のコースで内容を詳しく述べる

Geant4アプリケーション作成の第一歩

ユーザ・クラス実装の手順

- Geant4が提供するユーザ・クラスを使いシミュレーション実行コードを完成させる手順は以下の通り:

1. 以下の3つユーザ・クラスを継承して、自分のクラスを作成し、シミュレーションの初期条件を設定

- 1) 構造物 → *G4VUserDetectorConstruction* (抽象クラス)
- 2) 粒子と物質の相互作用 → *G4VUserPhysicsList* (抽象クラス)
- 3) 事象初期条件 → *G4VUserPrimaryGeneratorAction* (抽象クラス)

2. 以下の2つの手法を用いてsimulation結果の取得方法を設定

- 1) ユーザ・フックを使う → *G4UserRun(/Event/Tracking/Stepping)Action*
- 2) Sensitive Detector → *G4VSensitiveDetector* (抽象クラス)

3. 上で用意したクラスを用いてシミュレーション実行のmain()プログラムを書く

Geant4は完成したシミュレーション実行コードを提供するのではなく、C++で記述されたツールキットである

→ シミュレーションのフレームワークはユーザが自由に作成できる

[注] 以上の話を聞くと、アプリケーションを作るのは大変という印象を持つかもしれないが、本講座でGeant4の基礎を学べば実は簡単だといことが分かるであろう

→ Hands-onの例題のコードを実習してみることがベスト

Geant4の入出力ストリーム

- `G4cout`と`G4cerr`はGeant4が提供する出力ストリーム:

```
G4cout << "Hello Geant4!" << G4endl;    // G4endlも使える
G4cerr << "Error!" << G4endl;
```

- `G4cin`はGeant4が提供する入力ストリーム:

Geant4アプリケーションではC++クラス・ライブラリの`std::cin`ではなく、`G4cin`を使うことをすすめる

- Geant4のGUIのいくつかは上記出力ストリームを通して出力値をバッファリングし、画面表示・保存・編集に用いる

したがって、ユーザはC++クラス・ライブラリの`std::cout`及び`std::cerr`をGeant4アプリケーションでは**使うべきでない**

Geant4が用いる単位系

- Geant4が**内部的に保持する標準単位系**は、ユーザのみならずGeant4 toolkit自体のソースコードからも**あらわに見えない**ように設計されている
- そのかわりに、ユーザが単位を持つ変数を定義する時は、必ず単位を明記する

```
Radius = 10.0 * cm;
```

← 単位を **かけ算** で付与する

```
kineticE = 1.0 * GeV;
```

```
momentum = 5.0 * GeV;
```

← [注] 運動量は内部単位(次スライド参照)でMeV/cを採用している。従って5.0 GeV/cの運動量を表現したいならGeVをかけて5.0*GeVと書く

- 単位を持つ変数から物理値を取得する際は、必ず単位を明記する必要がある

```
G4cout << eDep / MeV << " [Mev]" << G4endl;
```

← 単位を **わり算** で付与する

- 通常使う単位 (cm, GeV, MeV等) は、Geant4が**標準で定数として提供**
 - ユーザは定数を使うだけでOK: 上のcm, GeV等は標準定義した定数
 - Geant4が提供する単位定数は“G4SystemOfUnits.hh” ファイルに定義されている
 - ユーザは定数を使用する際、このファイルを‘include’する
 - ユーザが独自の単位を定義する事も可能

[注] 物理量を扱う際は、必ず標準単位定数を常に使うこと:

- 物理量を扱う際の混乱を回避できる
- 外部ファイルへの入出力時の単位の混乱も避けられる

Geant4が用いる単位系 — 続き

- ユーザがもし単位を明示せずに物理変数を定義すると(これは絶対に避けるべきであるが)、その物理量は以下のGeant4内部単位系を持つものとして処理される:

長さ: mm	時間: ns	エネルギー: Mev	温度: Kelvin
物質質量: mole	光度: candela	角度: radian	立体角: steradian
電荷: positron charge			

- Geant4で標準的に使える単位の一覧を見るには以下のようにする

- UI コマンド: `Idle> /units/list`
- プログラムコード: `G4UnitDefinition::PrintUnitsTable();`

- 単位を持つ変数から物理値を取る時に、その値に応じて最適の単位を自動的にえらぶことができる

```
#include "G4UnitsTable.hh"
G4cout << G4BestUnit(eDep, "Energy") << G4endl;
```

← "Length", "Time", "Energy", "Volume", "Mass",
単位一覧の中のカテゴリ名

Geant4で使える物理定数

■ Geant4は標準的な物理定数を提供

```
G4double sectorAngle = twopi/nSector;  
G4double density = universe_mean_density;
```

[注] 上の例でtwopi、universe_mean_densityはGeant4が提供する物理定数

twopi = $2.0 \times 3.14159265358979323846$;

universe_mean_density = $1.e^{-25} \text{g/cm}^3$;

■ 提供されている定数の例

pi, twopi, halfpi、pi2、c_light、c_squared、h_Planck、hbar_Planck、hbarc、hbarc_squared、
electron_charge、e_squared、electron_mass_c2、proton_mass_c2、neutron_mass_c2、
elm_coupling、fine_structure_const、classic_electr_radius、electron_Compton_length、等

■ Geant4で標準的に備えている物理定数の一覧は以下のソース参照

[Geant4/externals/clhep/include/CLHEP/Units/PhysicalConstants.h](#)

■ ユーザが標準的な物理定数を使う場合、以下の宣言を".cc"ファイル中にする

```
#include "G4PhysicalConstants.hh"
```

ユーザアプリケーションのmain()プログラムの作成

■ 基本的な手続きは以下のようなになる:

- 1) main()の作成
- 2) **G4RunManager**オブジェクトを作成
- 3) ジオメトリ(**G4VUserDetectorConstruction**)記述オブジェクトをG4RunManagerに直接登録
- 4) 物理相互作用(**G4VUserPhysicsList**)記述オブジェクトをG4RunManagerに直接登録
- 5) 事象初期条件(**G4VUserPrimaryGeneratorAction**)記述オブジェクトをG4RunManagerに登録
 - **G4VUserActionInitialization**記述オブジェクトを通してG4RunManagerに登録
 - またはG4RunManagerに直接登録 (Geant4.10以前の手法、*ただしGeant4.10以降でもOK*)

```
//-----  
int main( int argc, char** argv )  
//-----  
{  
    // Construct the default run manager  
    G4RunManager * runManager = new G4RunManager{};  
  
    // Set up mandatory user initialization: Geometry  
    runManager->SetUserInitialization( new Geometry{} );  
  
    // Set up mandatory user initialization: Physics-List  
    runManager->SetUserInitialization( new FTFP_BERT{} );  
  
    // Set up user initialization: User Actions  
    runManager->SetUserInitialization( new UserActionInitialization{} );  
  
    // Initialize G4 kernel  
    runManager->Initialize();  
  
    .....  
}
```

三つの
必須作成クラスの
オブジェクト

[注]

ユーザが'new'で作ったrun manager
に渡したオブジェクトは、ユーザが
明示的に'delete'する必要はない

main()の詳細は
Hands-Onで学ぶ

Geant4 マルチスレッド・モードについて

■ Geant4のマルチスレッド対応

- Geant4 11.0 (Dec. 2021)からマルチスレッドによる並列処理に対応
- Geant4ライブラリをビルドすると、デフォルトでマルチスレッド対応
 - ➡ 本講習会で提供しているライブラリはマルチスレッド対応
- マルチスレッド対応Geant4ライブラリは常にシリアル処理にも対応

■ マルチスレッドを使うかシリアル処理を使うか？

- ユーザが自由に選択できる
 - ➡ シミュレーションの物理結果は同一
- 選択の指針に関しては本講習会のAppendix資料を参照
 - A01: マルチスレッド入門
 - A03: マルチスレッド化の具体的方法

■ 本講習会でシリアル処理を採用している理由

- マルチスレッド処理に伴うプログラムの複雑さが避けられる
- 採用しているmain()プログラムを含む全てのコードは、ユーザが今後作成するであろう実用的なシミュレーションの枠組みとして使用可能

講義 終了